Cours / TP : Découverte des Makefiles en C++

1. Rappel : compiler un programme C++

Sans Makefile, pour compiler un fichier `main.cpp` :

```
g++ -Wall -Wextra main.cpp -o programme
```

Pour exécuter :

```
./programme
```

Un Makefile permet d'automatiser ces opérations.

L'idée : au lieu de taper les commandes de compilation, on tape simplement :

```
make
./programme
```

Et le système exécute les bonnes commandes.

Ou directement :

```
make run
```

> La commande `make run` compile le programme si nécessaire, puis l'exécute automatiquement.

Structure d'un Makefile

Un Makefile contient des règles de la forme :

```
cible : dependances
<TAB> commande
```

- Cible : ce que l'on souhaite produire.
- Dépendances : les fichiers nécessaires pour produire la cible.
- Commande : l'action à effectuer.

Note : la commande doit commencer par une véritable tabulation et non des espaces.

2. Exemple 1 : programme dans un seul fichier

Nous allons reprendre l'exemple du tableau et calculer la somme de ses éléments.

```
projet1/
├── main.cpp
└── Makefile
```

Fichier `main.cpp`

```cpp
#include <iostream>
using namespace std;

int main() {
    int tableau[5] = {2, 4, 6, 8, 10};
    int somme = 0;

    for (int i = 0; i < 5; i++) {
        somme += tableau[i];
    }

    cout << "Somme = " << somme << endl;

    return 0;
}
```

Pour compiler le programme :

```
g++ -Wall -Wextra main.cpp -o programme
```

Pour l'exécuter :

```
./programme
```

Avec Makefile

On crée un fichier nommé exactement `Makefile` dans le même dossier.

```
CXX = g++
CXXFLAGS = -Wall -Wextra

programme: main.cpp
    $(CXX) $(CXXFLAGS) main.cpp -o programme

run: programme
    ./programme

clean:
    rm -f programme

.PHONY: run clean
```

Remarques :

- `CXX` désigne le compilateur C++.
- `CXXFLAGS` contient les options de compilation.
- `$(CXX)` et `$(CXXFLAGS)` permettent de réutiliser les variables.

Les commandes :

```
make        # compile
make run    # compile si nécessaire puis exécute
make clean  # supprime l'exécutable
```

Après compilation :

```
projet1/
├── main.cpp
├── Makefile
└── programme
```

---

3. Exemple 2 : programme avec séparation du code

On va maintenant séparer les fonctions dans différents fichiers.

Organisation du projet :

```
projet2/
├── main.cpp
├── tableau.cpp
├── tableau.h
└── Makefile
```

Fichier `tableau.h`

On déclare les fonctions.

```
#ifndef TABLEAU_H
#define TABLEAU_H

int somme(int tab[], int taille);
float moyenne(int tab[], int taille);

#endif
```

Fichier `tableau.cpp`

On implémente les fonctions.

```
#include "tableau.h"

int somme(int tab[], int taille) {
    int resultat = 0;

    for (int i = 0; i < taille; i++) {
        resultat += tab[i];
    }

    return resultat;
}

float moyenne(int tab[], int taille) {
    return static_cast<float>(somme(tab, taille)) / taille;
}
```

Fichier `main.cpp`

Le programme principal utilise les fonctions.

```cpp
#include <iostream>
#include "tableau.h"

using namespace std;

int main() {
    int tab[5] = {2, 4, 6, 8, 10};

    cout << "Somme : " << somme(tab, 5) << endl;
    cout << "Moyenne : " << moyenne(tab, 5) << endl;

    return 0;
}
```

Fichier `Makefile`

```
CXX = g++
CXXFLAGS = -Wall -Wextra

programme: main.o tableau.o
    $(CXX) $(CXXFLAGS) main.o tableau.o -o programme

main.o: main.cpp tableau.h
    $(CXX) $(CXXFLAGS) -c main.cpp

tableau.o: tableau.cpp tableau.h
    $(CXX) $(CXXFLAGS) -c tableau.cpp

run: programme
    ./programme

clean:
    rm -f *.o programme

.PHONY: run clean
```

4. Comprendre la compilation



L'intérêt du Makefile est notamment de ne recompiler que les fichiers qui ont été modifiés (ou dont les dépendances ont changé).

5. TP

Créer un programme C++ qui :

1. Initialise un tableau de 10 entiers.
2. Calcule le nombre de valeurs paires.
3. Calcule la somme des valeurs.
4. Affiche les résultats.
5. Utilise un Makefile pour compiler le programme.
6. Ajoute une cible `clean`.

Partie B – Séparation du code

```
tp_calculs/
├── main.cpp
├── calculs.cpp
├── calculs.h
└── Makefile
```

Reprendre le programme précédent et :

1. Créer un fichier `calculs.h`.
2. Créer un fichier `calculs.cpp`.
3. Déplacer les fonctions de calcul dans `calculs.cpp`.
4. Déclarer les prototypes dans `calculs.h`.
5. Adapter `main.cpp`.
6. Modifier le Makefile pour compiler séparément les fichiers.

Partie C – Questions de compréhension

1. Quel est le rôle de `CXX` ?
2. Quelle est la différence entre `main.cpp` et `main.o` ?
3. À quoi sert l'option `-c` ?
4. Pourquoi `tableau.h` apparaît-il dans les dépendances ?
5. Que se passe-t-il si on modifie uniquement `tableau.cpp` ?
6. Pourquoi utilise-t-on `make clean` ?

---

6. Synthèse

Notion	Signification
`Makefile`	Fichier décrivant les règles de compilation et d'exécution
`make`	Lance la construction de la cible par défaut
`make run`	Compile si nécessaire puis exécute le programme
`make clean`	Supprime les fichiers générés
Cible	Résultat à produire, par exemple `programme` ou `main.o`
Dépendance	Fichier nécessaire à la construction de la cible
`.cpp`	Fichier source C++
`.h`	Fichier d'en-tête contenant notamment les déclarations
`.o`	Fichier objet issu de la compilation
`.PHONY`	Déclare une cible qui ne correspond pas à un fichier réel